# Deep Dive Memory

Name: Rahul Vedant Pemsing
Student ID: 36305529
Date: 18/08/2025

## Learning Journey and Evidence

### Prior Knowledge and Experience

Before attempting this task, I had already built a strong foundation through both my previous studies and the earlier tasks (P1–P8).

From my A-level studies, I am well versed in linked list dynamic arrays, stack and queue. I have the knowledge of how it works but lack the knowledge on how to implement them properly as for A-levels, we implemented them in a large array imitating memory. We still use pointers so access the array location as if it was a memory location itself.

As for member function, I am familiar with it from the OOP study requirements for my A-levels.

Through P1–P7, I further developed my programming knowledge step by step:

- P1 (Learning Plan): Helped me set clear goals, organize my study approach, and identify areas where I needed to improve.
- P2 (Building and Running): Gave me practical skills of executing programs, as well as debugging common errors.
- P3 (Sequence and Data): Strengthened my understanding of variables, operators, and data handling, which is the basis of problem-solving experience.
- P4 (Control Flow): Taught me how to use conditionals and loops effectively to control the execution path of programs.
- P5 (C/C++): Reinforced syntax rules and highlighted the importance of precision in coding (e.g., C++ being case-sensitive).
- P6 (Structuring Code): Showed me how to modularize programs using functions and procedures, improving readability and reusability.
- P7 (Structuring Data): Introduced enums, structures, and the organization of data in memory, preparing me for more advanced concepts like pointers and references.

Together, these tasks gave me both the theoretical knowledge and practical experience to approach memory management.

## Study to Acquire, Refresh, and/or Extend Capability

To study and master this topic I did the following:

| 15/09/2025 - Read the field intensively |
| --- |
| **Description:**<br>Reading the field guide here is of utmost importance in my opinion as the concepts explained are essential to be able to code linked lists, dynamic arrays classes… |
| From this, I learned the following;<br>● Memory and the Heap<br>● C++ Memory Allocations<br>● Freeing memory<br>● Using Malloc,calloc and realloc<br>● Dynamic array<br><br>I have also refreshed my memory on the following topics;<br>● Linked list<br>● OOP<br>○ classes<br>○ objects<br>○ methods<br>○ inheritance<br>○ polymorphism |

15/09/2025 - Coding[Field Guide → Part 2 → 6.Memory deep dive → Guided Tour → Using Malloc]

**Description:**

I followed the field guide to get used to this foreign concept to me.

**My code:**

dynamic_test.cpp

dynamic > dynamic_test.cpp > ...

```cpp
#include <stdlib.h>
#include "splashkit.h"
#include "utilities.h"


int main(){

    double *data_ptr;
    int size;


    size = read_integer("What is the amout of data to be added: ");
    data_ptr = (double *)malloc(size * sizeof(double));

    if (data_ptr == NULL){
        write_line("Memory allocation failed");
        return 1;
    }

    for (int i =0 ; i < size; i++){
        data_ptr[i]= read_double("Enter value No. " + std::to_string(i) + ": ");
    }
    write_line("");
    write_line("======data dump=======");
    for (int i = 0; i<size; i++){
        write_line(data_ptr[i]);
    }

    free(data_ptr);
    data_ptr=NULL;
    return 0;
}

```

**Output:**

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/dynamic
$ ./d
What is the amout of data to be added: 12
Enter value No. 0: 10
Enter value No. 1: 15
Enter value No. 2: 12
Enter value No. 3: 14
Enter value No. 4: 18
Enter value No. 5: 16
Enter value No. 6: 18
Enter value No. 7: 16
Enter value No. 8: 13
Enter value No. 9: 17
Enter value No. 10: 18
Enter value No. 11: 16

======data dump=======
10.000000
15.000000
12.000000
14.000000
18.000000
16.000000
18.000000
16.000000
13.000000
17.000000
18.000000
16.000000

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/dynamic
$
```

From this, I learned how to implement malloc in my program although to a basic level. From this first step, I will be able to continue expanding my knowledge and gain the knowhow to implement it in a bigger program.

15/09/2025 - Coding[Field Guide → Part 2 → 6.Memory deep dive → Advance Tour → Dynamic Array]

**Description:**
An array is relatively straightforward to picture. It is a variable which contains multiple values. In C/C++, arrays have a fixed size. Now we know about memory management. We can build our own data type and functions to build a dynamic array, where its size can grow and shrink based on its usage.

**My code:**

My .hpp file:

dynamic_array.hpp

stats_calculator_dynamic > dynamic_array.hpp > ...

```
#include "splashkit.h"
#include "dynamic_array.h"
#include <new>
#include <cstdlib>



template<typename T>
d_arr<T> *new_dynamic_array(unsigned int capacity)
{
    d_arr<T> *result = (d_arr<T>*)malloc(sizeof(d_arr<T>));

    if (result == nullptr){
        return result;
    }
    result->data = (T*)malloc(sizeof(T)*capacity);
    result->size = 0;

    if (result->data == nullptr){
        result->capacity = 0;

    }
    else{
        result->capacity= capacity;

    }

    for (int i = 0; i < capacity; i++){
        new(&result->data[i]) T();
    }
    return result;
}
```

```cpp
template<typename T>
void delete_dynamic_array(d_arr<T> *array){
    if (!array) return;

    for (int i =0; i< array->capacity; i++){
        array->data[i].~T();
    }

    array->size = 0;
    array->capacity = 0;

    free(array->data);

    free(array);
}

template<typename T>
bool resize(d_arr<T> *arr, unsigned int new_capacity){
    if (!arr) return false;
    for (int i = arr->capacity - 1 ; i>=(int)new_capacity; i--){
        arr->data[i].~T();
    }

    T* new_data = (T*)realloc(arr->data, sizeof(T)* new_capacity);
    if(new_data == nullptr){
        return false;
    }

    arr->data = new_data;
    arr->capacity = new_capacity;

    if (new_capacity < arr->size){
        arr->size = new_capacity;
    }
    return true;
}
```

```
template<typename T>
bool add(d_arr<T> *array, T value){
    if (!array) return false;

    if (array->size >= array->capacity && !resize(array, array->capacity *2+1)){
        return false;
    }

    array->data[array->size]= value;
    array->size++;

    return true;
}

template <typename T>
T get(const d_arr<T> *array, unsigned int index){
    if (!array) throw string("Array is not allocated");

    if (index >= array->size){
        throw string ("Index out of bounds");
    }
    return array->data[index];
}


template<typename T>
void set(d_arr<T> * array, unsigned int index, T value){
    if (!array) throw string ("Array is not allocated");

    if (index >= array->size){
        throw string("Index out of bounds");
    }
    array->data[index]= value;
}
;

```

The header file:

stats_calculator_dynamic > C dynamic_array.h > ...

```cpp
#ifndef dynamic_array_h
#define dynamic_array_h

#include <string>
#include <cstdlib>
using std::string;

/**
 * @brief A dynamic array struct that contains the size, capacity,
 *        and data pointer used to implement this dynamic structure.
 *
 * @tparam T the type (typename) of data to store in the dynamic array
 * @field data a pointer to the data in the dynamic array on the heap
 * @field size the number of elements used in the dynamic array
 * @field capacity the number of elements the dynamic array can hold
 */
template<typename T>
struct d_arr
{
    T *data;
    unsigned int size;
    unsigned int capacity;
};

/**
 * @brief Create a new dynamic array with the indicated initial capacity.
 *        This will return a nullptr if the array cannot be allocated.
 *
 * @tparam T the type of data the array will store
 * @param capacity its initial capacity, with a default value of 50
 * @return dynamic_array<T>* a pointer to the new dynamic array
 */
template <typename T>
d_arr<T> *new_dynamic_array(unsigned int capacity = 50);
```

```cpp
/**
 * @brief Free the memory allocated to the dynamic array. Once freed
 *        the data in the array will no longer be accessible.
 *
 * @tparam T the data type of the dynamic array
 * @param array a pointer to the dynamic array to free
 */
template <typename T>
void delete_dynamic_array(d_arr<T> *array);

/**
 * @brief Resize the capacity of the dynamic array.
 *
 * If the new capacity is smaller than the current size, the size
 * will be updated to match the new capacity.
 *
 * @tparam T the type of data in the dynamic array
 * @param array the dynamic array to grow
 * @param new_capacity the new capacity of the dynamic array
 * @returns true if this succeeded, or false if it could not reallocate memory
 */
template <typename T>
bool resize(d_arr<T> *arr, unsigned int new_capacity);

/**
 * @brief Add an element to the end of the dynamic array
 *
 * @tparam T the type of data in the dynamic array
 * @param value the value to add to the end of the dynamic array
 * @param array the dynamic array to add the value to
 * @returns true if this succeeded, false if we cannot resize to fit the new element
 */
template <typename T>
bool add(d_arr<T> *array, T value);
```

```cpp
/**
 * @brief read and return the value of the indicated element from the dynamic array.
 *
 * If the array is not allocated or the index is out of bounds, the function will throw a string exception message.
 *
 * @tparam T the type of data in the dynamic array
 * @param array the dynamic array to remove the element from
 * @param index the index of the element to remove
 * @throws a string message if the array is not allocated or the index is invalid
 */
template <typename T>
T get(const d_arr<T> *array, unsigned int index);

/**
 * @brief set the value of the indicated element from the dynamic array.
 *
 * If the index is out of bounds, the function will throw an string exception.
 *
 * @tparam T the type of data in the dynamic array
 * @param array the dynamic array to set the element in
 * @param index the index of the element to change
 * @param value the value to set the element to
 */
template <typename T>
void set(d_arr<T> *array, unsigned int index, T value);


#include "dynamic_array.hpp"

#endif
```

Implemented it in to statistics calculator:

stats_calculator_dynamic > stats_dynamic.cpp > ...

```
#include "splashkit.h"
#include "dynamic_array.h"
#include <stdio.h>

/** function to read an integer greater than 1
 * @param prompt - passed by value, of data type string; contain the output message for the user to know what to input
 *
 * @return a positive interger greater than 0
 */
int read_integer(string prompt)
{
    write(prompt);
    string line = read_line();
    bool flag = false;
    int num;
    do
    {
        while (!is_integer(line))
        {
            write("Please enter number greater than 0: ");
            write(prompt);
            line = read_line();
        }
        num = stoi(line);
        if (num > 0)
        {
            flag = true;
        }

    } while (!flag);
    return num;
}
```

```

/** function to read an real(double) value
 * @param prompt passed by value, of data type string; contain the output message for the user to know what to input
 *
 * @return a double value
 */
double read_double(string prompt)
{
    write(prompt);
    string line = read_line();
    bool flag = false;
    double num;
    while (!is_double(line))
    {
        write("Please enter a number: ");
        write(prompt);
        line = read_line();
    }
    num = convert_to_double(line);
    return num;
}
```

```
/**
 * procedure to fill the array
 *
 * @param *data a pointer to the dynamic array
 */
void populate_array(d_arr<double> *data)
{
    int numdata = read_integer("How many values do you want to add: ");
    for (int i = 0; i < numdata; i++)
    {
        add(data, read_double("Enter value no. " + std::to_string(i + 1) + ": "));
    }
}

/**
 * procedure to print the array
 *
 * @param *data a pointer to the dynamic array
 */
void print_array(const d_arr<double> *data)
{
    for (int i = 0; i < (int)data->size; i++)
    {
        printf("%d: %lf\n", i, get(data, i));
    }
}
```

```
/**
 * function for return the maximum
 *
 * @param *data a pointer to the dynamic array
 *
 * @return maximum - a double value
 */
double maximum(const d_arr<double> *data)
{
    double max_val = get(data, 0);
    for (int i = 1; i < (int)data->size; i++){
        double val = get(data, i);
        if (val > max_val){
            max_val = val;
        }
    }
    return max_val;
}

/**
 * function for return the sum
 *
 * @param *data a pointer to the dynamic array
 *
 * @return sum - a double value
 */
double sum(const d_arr<double> *data)
{
    double total = 0;
    for (int i = 0; i < (int)data->size; i++){
        total += get(data, i);
    }
    return total;
}
```

```c
/**
 * function for return the mean
 *
 * @param *data a pointer to the dynamic array
 *
 * @return mean - a double value
 */
double mean(const d_arr<double> *data)
{
    if (data->size == 0)
        return 0;
    return sum(data) / data->size;
}

/**
 * procedure for print an element of the dynamics array
 *
 * @param *data a pointer to the dynamic array
 *
 */
void print(const d_arr<double> *data)
{
    for (int i = 0; i < data->size; i++)
    {
        printf("%d: %lf\n", i, get(data, i));
    }
}
```

```c

/**
 * procedure for add data to the dynamic array
 *
 * @param *data a pointer to the dynamic array
 *
 */
void add_data(d_arr<double> *data)
{
    double value = read_double("Enter a value to add: ");
    if (!add(data, value))
    {
        printf("Sorry, out of memory\n");
    }
}
```

```c
/**
 * procedure to remove an elenent from the dynamic array
 *
 * @param *data a pointer to the dynamic array
 *
 */
void remove_data(d_arr<double> *data)
{
    print(data);

    int index = read_integer("Enter the index of the value to remove: ");

    if (index >= 0 && index < data->size)
    {
        for (int i = index; i < data->size - 1; i++)
        {
            set(data, i, get(data, i + 1));
        }
        resize(data, data->size - 1);
    }
    else
    {
        printf("Sorry, that is not a valid indez.\n");
    }
}
```

```c
int main()
{
    d_arr<double> *data = new_dynamic_array<double>(10);

    populate_array(data);
    print_array(data);

    double max_val = maximum(data);
    double total = sum(data);
    double avg = mean(data);

    printf("\nThe maximum value is %.2f, the sum is %.2f, and the mean is %.2f.\n", max_val, total, avg);

    add_data(data);
    print(data);
    write_line("");
    remove_data(data);
    write_line("");
    print(data);

    delete_dynamic_array(data);
    data = nullptr;

    return 0;
}
```

**Output:**

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/stats_calculator_dynamic
$ ./s2
How many values do you want to add: 5
Enter value no. 1: 12.5
Enter value no. 2: 13
Enter value no. 3: 14.5
Enter value no. 4: 18.6
Enter value no. 5: 17.3
0: 12.500000
1: 13.000000
2: 14.500000
3: 18.600000
4: 17.300000

The maximum value is 18.60, the sum is 75.90, and the mean is 15.18.
Enter a value to add: 21.7
0: 12.500000
1: 13.000000
2: 14.500000
3: 18.600000
4: 17.300000
5: 21.700000

0: 12.500000
1: 13.000000
2: 14.500000
3: 18.600000
4: 17.300000
5: 21.700000
Enter the index of the value to remove: 2

0: 12.500000
1: 13.000000
2: 18.600000
3: 17.300000
4: 21.700000

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/stats_calculator_dynamic
```

From this exercise, I learned how to implement a dynamic array and integrate it into a real program (the statistics calculator). I now understand how memory can be allocated and resized at runtime, and how this allows a program to handle flexible amounts of data instead of being limited by fixed array sizes. It showed me how important memory management is in real-world applications.

16/09/2025 - Coding[Field Guide → Part 2 → 6.Memory deep dive → Advance Tour → Linked List]

**Description:**

A Linked List is a term given to a certain way data can be structured in memory.
A Linked List has Nodes, the equivalent of the elements of an array.
Each Node has some data and a pointer to the Next element in the list.
The Last element in the list has Nothing as its next node.
To access a Node in the list you must loop through from the first node until you reach the node you are after.
You can insert and delete elements by changing the links in the list.

**My code:**

```
#include <stdio.h>
#include <stdlib.h>
#include <new>

/**
 * @brief A node in the linked list.
 *
 * Each node stores a value of type T and a pointer to the next node.
 *
 * @tparam T the type of data stored in the node
 * @field data the stored data of type T
 * @field next a pointer to the next node in the list
 */
template <typename T>
struct node
{
    node *next;
    T data;
};

/**
 * @brief A singly linked list structure.
 *
 * The list maintains a pointer to the first and last nodes
 * to support efficient insertion at the end.
 *
 * @tparam T the type of data stored in the list
 * @field first pointer to the first node of the list
 * @field last pointer to the last node of the list
 */
template <typename T>
struct linked_list
{
    node<T> *first;
    node<T> *last;
};
```

```
/**
 * @brief Create a new empty linked list.
 *
 * Allocates memory for a new linked_list and sets
 * its first and last pointers to nullptr.
 *
 * @tparam T the type of data stored in the linked list
 * @return linked_list<T>* a pointer to the newly created list
 */
template <typename T>
linked_list<T> *new_linked_list()
{
    linked_list<T> *list = (linked_list<T> *)malloc(sizeof(linked_list<T>));
    list->first = nullptr;
    list->last = nullptr;
    return list;
}
```

```

/**
 * @brief Add a new node with the given data to the end of the list.
 *
 * A new node is created and appended to the list. If the list
 * is empty, the new node becomes both the first and last node.
 *
 * @tparam T the type of data stored in the linked list
 * @param list the list to add the node to
 * @param data the value to store in the new node
 */
template <typename T>
void add_node(linked_list<T> *list, T data)
{
    node<T> *new_node = (node<T> *)malloc(sizeof(node<T>));

    new (&new_node->data) T();

    new_node->data = data;
    new_node->next = nullptr;

    if (list->first == nullptr)
    {
        list->first = new_node;
        list->last = new_node;
    }
    else
    {
        list->last->next = new_node;
        list->last = new_node;
    }
}
```

```cpp
/**
 * @brief Delete the entire linked list and free its memory.
 *
 * Traverses the list, destroys each node's data, and frees
 * the memory allocated for each node. Finally, the list itself
 * is freed and its pointers reset to nullptr.
 *
 * @tparam T the type of data stored in the linked list
 * @param list the linked list to delete
 */
template <typename T>
void delete_lined_list(linked_list<T> *list)
{
    node<T> *current = list->first;
    while (current != nullptr)
    {
        node<T> *next = current->next;

        current->data.~T();
        current->next = nullptr;
        current = next;
    }

    list->first = nullptr;
    list->last = nullptr;
    free(list);
}
```

```cpp
/**
 * @brief Remove a specific node from the linked list.
 *
 * Adjusts the previous node's pointer to bypass the node,
 * updates the list's first/last pointers if necessary,
 * destroys the node's data, and frees its memory.
 *
 * @tparam T the type of data stored in the linked list
 * @param list the linked list to remove the node from
 * @param node_to_remove the node to remove
 */
template <typename T>
void remove_node(linked_list<T> *list, node<T> *node_to_remove)
{

    node<T> *privious = nullptr;

    if (list->first == node_to_remove)
    {
        list->first = node_to_remove->next;
    }
    else
    {
        node<T> *current = list->first;
        while (current != node_to_remove && current != nullptr)
        {
            privious = current;
            current = current->next;
        }

        if (current == nullptr)
            return;

        privious->next = node_to_remove->next;
    }

    if (list->last == node_to_remove)
    {
        list->last = privious;
    }

    node_to_remove->next = nullptr;
    node_to_remove->data.~T();
    free(node_to_remove);
}
```

```
/**
 * @brief Demonstration of linked list usage.
 *
 * Creates a linked list of integers, adds elements,
 * prints them, removes one element, prints again,
 * and deletes the list.
 */
int main()
{
    linked_list<int> *list = new_linked_list<int>();
    add_node(list, 1);
    add_node(list, 2);
    add_node(list, 3);
    add_node(list, 4);

    node<int> *current = list->first;
    while (current != nullptr)
    {
        printf("%d->", current->data);
        current = current->next;
    }
    printf("\n");

    remove_node(list, list->first->next->next);

    current = list->first;
    while (current != nullptr)
    {
        printf("%d->", current->data);
        current = current->next;
    }
    printf("\n");

    delete_lined_list(list);
}
```

**Output:**

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/linked_list
$ ./lkl
1->2->3->4->
1->2->4->
```

Through this task, I learned how linked lists store data in nodes connected by pointers. Unlike arrays, they are not stored contiguously in memory, which allows efficient insertion and deletion at any position. However, I also realized that accessing elements requires traversal from the start, which is slower than arrays. This helped me appreciate when linked lists are preferable to arrays.

16/09/2025 - Coding[Field Guide → Part 2 → 7.Member function → Guided tour → Transaction]

**Description:**
A simple bank account class.

**My code:**

```cpp
#include "splashkit.h"

/**
 * @brief Represents a bank account with a name and balance.
 *
 * Provides methods to print the account, deposit money, and withdraw money.
 */
class bank_account{
private:
    double balance;
public:
    string name;

    /**
     * @brief Construct a new bank account with a given name and initial balance.
     *
     * @param name the account holder's name
     * @param balance the initial balance of the account
     */
    bank_account(string name, double balance){
        this->name = name;
        this->balance = balance;
    }

    /**
     * @brief Construct a new bank account with default values.
     *
     * Default name is "Unknown", balance is 0.
     */
    bank_account(){
        name = "Unknown";
        balance = 0;
    }
```

```
    /**
     * @brief Print the account name and balance.
     */
    void print(){
        write_line(name + ": $" + std::to_string(balance));
    }

    /**
     * @brief Deposit money into the account.
     *
     * @param amount the amount to deposit
     */
    void deposit(double amount){
        balance += amount;
        write_line("$" + std::to_string(amount) + " has been deposited");
    }

    /**
     * @brief Withdraw money from the account.
     *
     * If there is not enough balance, the withdrawal will fail.
     *
     * @param amount the amount to withdraw
     */
    void withdraw(double amount){
        if (balance >= amount){
            balance -= amount;
            write_line("$" + std::to_string(amount) + " has been withdraw");
        }
        else{
            write_line("Not enought money...");
        }
    }
};
```

```cpp
/**
 * @brief Base class for transactions performed on a bank account.
 *
 * Defines the common interface and state for all transactions,
 * such as whether it has been performed, whether it succeeded,
 * and the transaction amount.
 */
class transaction{
    protected:
        bool performed = false;
        bool succeeded = false;
        double amount;
        bank_account *d_account;

    public:


        /**
         * @brief Construct a new transaction.
         *
         * @param d_acc pointer to the bank account
         * @param a the transaction amount
         */
        transaction(bank_account *d_acc, double a){
            this->d_account = d_acc;
            this->amount = a;
        }
```

```cpp
        /**
         * @brief Perform the transaction.
         *
         * Must be overridden by derived classes.
         */
        virtual void perform() = 0;

        /**
         * @brief Check whether the transaction has been performed.
         *
         * @return true if the transaction has been performed
         * @return false otherwise
         */

        bool was_performed() const{
            return performed;
        }

        /**
         * @brief Check whether the transaction was successful.
         *
         * @return true if the transaction succeeded
         * @return false otherwise
         */
        bool was_successful() const{
            return succeeded;
        }
    };
```

```cpp
/**
 * @brief A withdrawal transaction that deducts money from a bank account.
 */
class widthdraw_transaction : public transaction{
    public:

        /**
         * @brief Construct a new withdraw transaction.
         *
         * @param acc pointer to the bank account
         * @param a the amount to withdraw
         */
        widthdraw_transaction(bank_account *acc, double a) : transaction(acc, a) {}

        /**
         * @brief Perform the withdrawal.
         *
         * If already performed, throws an error string.
         * Attempts to withdraw money from the account.
         *
         * @throws string if the transaction has already been performed
         */
        void perform() override{
            if (performed){
                string error_msg = "Transaction already performed";
                throw error_msg;
            }
```

```
            try
            {
                d_account->withdraw(amount);
                succeeded = true;
            }
            catch(const string &e)
            {
                write_line("Transaction failed: " + e);
                succeeded = false;
            }
            performed = true;
        }
    };
```

```
/**
 * @brief A deposit transaction that adds money to a bank account.
 */
class deposit_transaction : public transaction{
    public:
        /**
         * @brief Construct a new deposit transaction.
         *
         * @param acc pointer to the bank account
         * @param a the amount to deposit
         */
        deposit_transaction(bank_account *acc, double a) : transaction(acc,a) {}
```

```
        /**
         * @brief Perform the deposit.
         *
         * If already performed, throws an error string.
         * Attempts to deposit money into the account.
         *
         * @throws string if the transaction has already been performed
         */
        void perform() override{
            if (performed){
                string err_msg= "Transaction already performed" ;
                throw err_msg;
            }

            try {
                d_account->deposit(amount);
                succeeded = true;
            }
            catch(const string &e)
            {
                write_line("Transaction failed " + e);
                succeeded = false;
            }
            performed = true;
        }
};
```

```cpp
int main(){
    bank_account acc("sam", 90125);

    transaction *test;

    acc.print();

    test = new widthdraw_transaction(&acc, 1000);

    test->perform();

    if (test->was_successful()){
        write_line("Transaction successful!");
    }
    else {
        write_line("Transaction failed");
    }

    acc.print();

    test = new deposit_transaction(&acc, 2000);
    test->perform();

    if (test->was_successful()){
        write_line("Transaction successful!");
    }
    else{
        write_line("Transaction failed");
    }
    acc.print();
    return 0;
}
```

**Output:**

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/pre_oop_account
$ ./a3
sam: $90125.000000
$1000.000000 has been withdraw
Transaction successful!
sam: $89125.000000
$2000.000000 has been deposited
Transaction successful!
sam: $91125.000000
```

This task taught me how to implement classes and member functions in C++. I understood how encapsulation works by grouping data (like account balance) with

methods (like deposit and withdraw). It also reinforced the importance of object-oriented principles for organizing and protecting data in real-world scenarios.

17/09/2025 - Coding[Field Guide → Part 2 → 7. Member function → Advance Tour → Dynamic Array V2]

**Description:**
Improving upon the prior version of dynamic array by making it look more like an array.

**My code:**

My .hpp file:

```
#ifndef DYNAMIC_ARRAY_H
#define DYNAMIC_ARRAY_H
#include <cstdlib>


/**
 * @brief A dynamic array struct taht contains the size, capcity, and data pointer used to implement thsi dynamic structure.
 *
 * @tparam T the type of data to store in the dynamic array
 * @field data a pointer to the data in the dynamic arrayon the heap
 * @field size the number of elements used in the dynamic array
 * @field capacity the number of elements the dynamic array can hold
 * @field default_value to use when getting an element
 */
template<typename T>
struct dynamic_array
{
    unsigned int size;
    unsigned int capacity;
    T *data;
    T default_value;

```

```cpp
    /**
     * @brief Create a new dynamic array with the indicated initial capacity.
     *
     * @param capacity its initial capacity
     * @param default_value the value to return if the index is out of bounds
     */
    dynamic_array(unsigned int capacity, T default_value){
        data = (T *)malloc(sizeof(T) * capacity);
        size = 0;
        this->default_value = default_value;

        if (data == nullptr){
            this->capacity = 0;
        }
        else{
            this->capacity = capacity;
        }

        for (int i = 0; i < capacity; i++){
            new(&this->data[i]) T();
        }
    }
```

```cpp
    /**
     * @brief free the memory allocated to the dynamic array. once freed the data in the array will no longer be acceddible
     */
    ~dynamic_array(){
        size = 0;

        for (int i = 0; i < capacity; i++){
            data[i].~T();
        }
        capacity = 0;

        free(data);
        data = nullptr;
    }
```

```cpp
    /**
     * @brief resize the capacity of the dynamic array
     *
     * If the new capacity is smaller than the current size, the size will be updated to match the new capacity
     *
     * @param new_capacity the new capacity of the dynamic array
     * @return true if this succeeded, or false if it could not reallocate memory
     */
    bool resize(unsigned int new_capacity){
        for(int i = capacity -1; i >=(int)new_capacity; i--){
            data[i].~T();
        }

        T *new_data = (T *)realloc(data, sizeof(T) * new_capacity)

        if (new_data == nullptr){
            return false;
        }

        for (int i = capacity; i < new_capacity; i++){
            new(&new_data[i]) T();
        }

        data = new_data;
        capacity = new_capacity;

        if (new_capacity < size){
            size = new_capacity;
        }
        return true;
    }
```

```cpp
    /**
     * @brief Add an element to the end of the dynamic array
     *
     * @param value the value to the add to the end of the dynamic
     */
    bool add(T value){

        if (size >= capacity && !resize(capacity * 2+1))
        {
            return false;
        }

        data[size] = value;
        size++;

        return true;
    }
```

```cpp

    /**
     * @brief read and return the value of the indicated element from the dynamic array.
     *
     * if the index is out of bounds, the function will return the indicated default value
     *
     * @param index the index of the element to remove
     */
    T &get(unsigned int index){
        return (*this)[index];
    }


    /**
     * provide array style access to the dynamic array
     *
     * @param index the index of the element to get
     * @return T& the element at the given index
     */
    const T &operator[](unsigned int index) const{
        if (index >= size){
            return default_value;
        }
        return data[index];
    }
```

```cpp
    /**
     * provide array style access to the dynamic array
     *
     * @param index the index of the element to get
     * @return T& the element at the given index
     */
    T &operator[](unsigned int index){
        if (index >= size){
            return default_value;
        }
    return data[index];
    }

    /**
     * @brief set the value of the indicated element from the dynamic array
     *
     * if the index is out of bounds, the function will do nothings and return false
     *
     * @param index the index of the element to change
     * @param value the value to set the element to
     * @return true when the value is set, or false if this failed
     */
    bool set(unsigned int index, T value){
        if(index >= size){
            return false;
        }

        data[index] = value;
        return true;
    }
};

#endif
```

Implementation in stats calculator:

```cpp
#include <cstdio>
#include <cstdlib>
#include "utilities.h"
#include "d_arr_v2.hpp"

/**
 * Populate the array with values entered by the user
 *
 * @param data the array the array of values (passed by refrences)
 */
void populate_array(dynamic_array<double> &data)
{

    for (int i = 0; i < data.size; i++)
    {
        data[i] = read_double("Enter value: ");
    }
}

/**
 * output the values in the array
 *
 * @param data the array of values
 */
void print(const dynamic_array<double> &data)
{
    for (int i = 0; i < data.size; i++)
    {
        printf("%d: %lf\n", i, data[i]);
    }
}
```

```
/**
 * calculate the sum of the values in the array
 *
 * @param data the array of values
 * @return the sum of the values
 */
double sum(const dynamic_array<double> &data)
{
    int i;
    double result = 0;

    for (i = 0; i < data.size; i++)
    {
        result += data[i];
    }
    return result;
}

/**
 * calculate the mean of the values in the array
 *
 * @param data the array of values
 * @returns the mean of the values
 */
double mean(const dynamic_array<double> &data)
{
    if (data.size > 0)
    {
        return sum(data) / data.size;
    }
    else
    {
        return 0;
    }
}
```

```
/**
 * Calculate the largest value in the array
 *
 * @param data the array of values
 * @returns the largest value, or 0 if there are no values
 */
double max(const dynamic_array<double> &data)
{
    if (data.size == 0)
    {
        return 0;
    }

    double result = data[0];

    for (int i = 1; i < data.size; i++)
    {
        if (data[i] > result)
        {
            result = data[i];
        }
    }
    return result;
}

/**
 * add a value from the array
 *
 * @param data the array of values
 */
void add_data(dynamic_array<double> &data)
{
    double value = read_double("Enter a value to add: ");
    data.add(value);
}
```

```
/**
 * remove a value from the array
 *
 * @param data the array of values
 */
void remove_value(dynamic_array<double> &data)
{
    print(data);

    int index = read_integer("Enter the index of the values to remove: ");

    if (index >= 0 && index < data.size)
    {
        for (int i = index; i < data.size - 1; i++)
        {
            data[i] = data[i + 1];
        }
        data.size--;
    }
    else
    {
        printf("Sorry, that is not a valid index\n");
    }
}
```

```c
/**
 * print the menu of option
 */
void print_menu()
{
    printf("\nMenu\n");
    printf("1. Add a value\n");
    printf("2. Remove a value\n");
    printf("3. Print the value\n");
    printf("4. Calculte stats\n");
    printf("5. Quit\n");
}

/**
 * The main entry point
 */
int main()
{
    dynamic_array<double> data = {{}, 0};
    int option;

    populate_array(data);

    print_menu();
    option = read_integer_limit("Enter an option: ", 1, 5);

    while (option != 5)
    {
        switch (option)
        {
        case 1:
            add_data(data);
            break;
```

```
        case 2:
            remove_value(data);
            break;

        case 3:
            printf("\nYou entered... \n\n");
            print(data);
            break;

        case 4:
            printf("\nCalculating statistics...\n\n");
            printf("sum:   %4.2lf\n", sum(data));
            printf("mean:  %4.2lf\n", mean(data));
            printf("Max:   %4.2lf\n", max(data));
            break;

        case 5:
            return 0;
            break;
        }
        read_string("Press enter to continue...");

        print_menu();
        option = read_integer_limit("Enter an option: ", 1, 5);
    }

    return 0;
}
```

**Output:**

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/stats_revisited
$ ./s

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculte stats
5. Quit
Enter an option: 1
Enter a value to add: 12
Press enter to continue...

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculte stats
5. Quit
Enter an option: 3

You entered...

0: 12.000000
Press enter to continue...

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculte stats
5. Quit
Enter an option: 1
Enter a value to add: 15.3
Press enter to continue...
```

```
Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculte stats
5. Quit
Enter an option: 1
Enter a value to add: 17.5
Press enter to continue...

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculte stats
5. Quit
Enter an option: 3

You entered...

0: 12.000000
1: 15.300000
2: 17.500000
Press enter to continue...

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculte stats
5. Quit
Enter an option: 4

Calculating statistics...
```

```
sum:   44.80
mean:  14.93
Max:   17.50
Press enter to continue...

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculte stats
5. Quit
Enter an option: 5

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/stats_revisited
$
```

By improving the dynamic array, I learned how to design a more user-friendly interface that behaves like a normal array. This exercise deepened my understanding of operator overloading and class design, making the custom data

structure easier to use. It also showed me how iterative refinement improves both usability and maintainability.

18/09/2025 - Coding[Field Guide → Part 2 → 7. Member function → Activities → Test yourself → User account]

**Description:**
Create a class to represent a user account. This should have a username and a password. The password should be kept private, so that it cannot be read outside the class. The system will have a single user, which is set when the program starts. Allow the user to test the login, and to change their password. The change password method should accept the current password, and the new password, and only update the password if the existing passwords match. The code should also check if the new password was not the previous password. You may choose to add additional constraints on the password as well.

Template output:

```
Username: Glory
Password: 123Er
- Account set.

1: Set account
2: Test login
3: Change password
4: Quit
> 2
Username: Glory
Password: abc
- Login failed

1: Set account
2: Test login
3: Change password
4: Quit
> 3
Current Password: 123er
New Password: abc123
- Failed to change password - invalid password

1: Set account
2: Test login
3: Change password
4: Quit
> 3
Current Password: 123Er
New Password: abc123
- Password changed

1: Set account
2: Test login
3: Change password
4: Quit
> 3
Current Password: abc123
New Password: 123Er
- Failed to change password - password reuse
```

**My code:**

user_account > user_acc.cpp > ...

```cpp
#include "splashkit.h"
#include <stdio.h>

/**
 * @brief Prompt the user with a message and return their string input.
 *
 * @param prompt the message to display before input
 * @return string the user input read from the console
 */
string read_string(string prompt)
{
    write(prompt);
    return read_line();
}
```

```cpp
/**
 * @brief Prompt the user to enter an integer within a specified range.
 *
 * Keeps asking until the input is valid and within the range [min, max].
 *
 * @param prompt the message to display before input
 * @param min the minimum acceptable integer
 * @param max the maximum acceptable integer
 * @return int a valid integer between min and max
 */
int read_integer_limit(string prompt, int min, int max)
{
    string line = read_string(prompt);
    int temp = min - 1;
    while (temp < min || temp > max)
    {
        while (!is_integer(line))
        {
            write_line("Please enter a integer number in the correct range.");
            line = read_string(prompt);
        }
        temp = stoi(line);
    }

    return temp;
}
```

```
/**
 * @brief Represents an account with a username and password.
 *
 * The class stores the current password, a history of the last 3
 * used passwords (to prevent reuse), and provides methods for
 * login testing and password management.
 */
class acc_details
{
private:
    string password;
    string used_password[3];

public:
    string username;

    /**
    * @brief Construct a new account with the given username and password.
    *
    * @param user_name the username to set
    * @param pass_word the initial password
    */
    acc_details(string user_name, string pass_word)
    {
        this->username = user_name;
        this->password = pass_word;
        printf("- Account set.\n");
    }
```

```
    /**
    * @brief Prompt the user for username and password, and check login validity.
    *
    * If the input matches the stored username and password, login is successful.
    * Otherwise, login fails.
    */
    void test_login()
    {
        string in_username = read_string("Username: ");
        string in_password = read_string("Password: ");

        if (in_username == username && in_password == password)
        {
            printf("- Login successful\n");
            return;
        }
        else
        {
            printf("- Login failed\n");
        }
    }
```

```
    /**
     * @brief Change the account password after validating credentials.
     *
     * Checks:
     * - Username must match the stored username.
     * - Current password must match.
     * - New password must not be one of the last 3 used.
     *
     * If successful, the password is updated, and the password history is shifted.
     */
    void change_password()
    {
        string in_username = read_string("Username: ");
        string in_password = read_string("Current password: ");
        string new_password = read_string("New password: ");
        bool flag = false;

        for (int i = 0; i < 3; i++)
        {
            if (new_password == used_password[i])
            {
                flag = true;
            }
        }
```

```
        }

        if (!(in_username == username))
        {
            printf("- Failed to change password - invalid username\n");
            return;
        }
        else if (!(in_password == password))
        {
            printf("- Failed to change password - invalid password\n");
            return;
        }
        else if (flag)
        {
            printf("- Failed to change password - password reuse\n");
        }
        else
        {

            for (int i = 1; i < 3; i++)
            {
                used_password[i - 1] = used_password[i];
            }
            used_password[2] = password;
            password = new_password;
            printf("Password changed successfully!\n");
        }
    }
};
```

```c
/**
 * @brief Display the account management menu options.
 */
void menu()
{
    printf("\n1: Set account");
    printf("\n2: Test login");
    printf("\n3: Change password");
    printf("\n4: Quit");
}
```

```c
/**
 * @brief Main entry point of the program.
 *
 * Provides a simple account management system that:
 * - Creates an account
 * - Allows login testing
 * - Allows password changes with history checks
 *
 * Runs in a loop until the user chooses to quit.
 */
int main(){
    acc_details account(read_string("Username: "), read_string("Password: "));

    int choice;
    do{
        menu();
        printf("\n");
        choice = read_integer_limit("> ", 1, 4);

        switch (choice){
        case 1:
            account = acc_details(read_string("Username: "), read_string("Password: "));
        case 2:
            account.test_login();
            break;

        case 3:
            account.change_password();
            break;

        case 4:
            break;
        }

    } while (choice != 4);
}
```

**Output:**

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/user_account
$ ./u
Username: sam
Password: sam002
- Account set.

1: Set account
2: Test login
3: Change password
4: Quit
> 2
Username: sam
Password: sam003
- Login failed

1: Set account
2: Test login
3: Change password
4: Quit
> 2
Username: sam1
Password: sam002
- Login failed

1: Set account
2: Test login
3: Change password
4: Quit
> 2
Username: sam
Password: sam002
- Login successful
```

```
- Login successful

1: Set account
2: Test login
3: Change password
4: Quit
> 3
Username: sam
Current password: sam002
New password: sam001
Password changed successfully!

1: Set account
2: Test login
3: Change password
4: Quit
> 3
Username: sam
Current password: sam001
New password: sam003
Password changed successfully!

1: Set account
2: Test login
3: Change password
4: Quit
> 3
Username: sam
Current password: sam003
New password: sam002
- Failed to change password - password reuse

1: Set account
2: Test login
3: Change password
4: Quit
> 4
Account has been deleted
```

Here I learned how to implement encapsulation to protect sensitive data (like passwords). I gained practical experience in designing secure methods for password validation and updates. This task highlighted the importance of data privacy, security checks, and avoiding common vulnerabilities when dealing with user authentication.

18/09/2025 - Coding[Field Guide → Part 2 → 7. Member function → Activities → Test yourself → Dice]

**Description:**

Create a class to represent a die (a single dice). This can have a number of sides, which cannot change after it is created. The die will also have a current value, being the value visible on top when the die is rolled. The die can be rolled, which randomises the value shown on top based on the number of sides. The current value can be read, but not changed by other code. Include a method to print the details of the die, including its sides and current value so that it can be shown in the test code.

**My code:**

dice > dice.cpp > ...

```cpp
#include "splashkit.h"

/**
 * @brief Read a line of input from the user as a string.
 *
 * Prompts the user with a given message and returns the input as a string.
 *
 * @param prompt The message displayed to the user
 * @return The string entered by the user
 */
string read_string(string prompt)
{
    write(prompt);
    return read_line();
}
```

```cpp
/**
 * @brief Read an integer from the user.
 *
 * Keeps prompting until the user enters a valid integer.
 *
 * @param prompt The message displayed to the user
 * @return The integer entered by the user
 */
int read_integer(string prompt){
    string line = read_string(prompt);

    while (!is_integer(line))
    {
        write_line("Please enter a integer number.");
        line = read_string(prompt);
    }
    return stoi(line);
}
```

```
/**
 * @brief Read an integer within a given range from the user.
 *
 * Keeps prompting until the user enters a valid integer between min and max.
 *
 * @param prompt The message displayed to the user
 * @param min The minimum valid value
 * @param max The maximum valid value
 * @return The integer entered by the user, guaranteed to be in [min, max]
 */
int read_integer_limit(string prompt, int min, int max)
{
    string line = read_string(prompt);
    int temp = min - 1;
    while (temp < min || temp > max)
    {
        while (!is_integer(line))
        {
            write_line("Please enter a integer number in the correct range.");
            line = read_string(prompt);
        }
        temp = stoi(line);
    }

    return temp;
}
```

```
/**
 * @brief Represents a die (dice) with a given number of faces.
 *
 * Provides methods to roll the die and print its current value.
 */
class dice
{
private:
    int face;
    int current_value;
public:

    /**
     * @brief Construct a new dice object with a specified number of faces.
     *
     * @param die_face The number of sides of the die
     */
    dice(int die_face){
        this->face = die_face;
    }

    /**
     * @brief Print the die's number of sides and its current rolled value.
     */
    void print_die(){
        printf("%d sides: value is %d\n", face, current_value);
    }
```

```c
    /**
     * @brief Roll the die to generate a new random value.
     *
     * Uses splashkit's rnd function to generate a random number
     * between 1 and the number of sides of the die.
     */
    void roll_die(){
        current_value = rnd(1,face);
    }
};

/**
 * @brief Display the program menu options to the user.
 */
void menu(){
    printf("\n1: Roll die\n");
    printf("2: Get new die\n");
    printf("3: Quit\n");
}
```

```
/**
 * @brief Main program entry point.
 *
 * Creates a die with user-specified sides, then provides
 * a menu to roll the die, create a new die, or quit.
 *
 * @return Exit status code
 */
int main(){
    dice die(read_integer("Sides: "));

    int choice;

    do
    {
        menu();
        choice = read_integer_limit("> ", 1, 3);

        if (choice == 1){
            die.roll_die();
            die.print_die();
        }
        else if (choice == 2)
        {
            die = dice(read_integer("Sides: "));
        }

    } while (choice != 3);

    return 0;
}
```

**Output:**

```
Vedant@LAPTOP-Q7B9365O MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/dice
$ ./d2
Sides: 6

1: Roll die
2: Get new die
3: Quit
> 1
6 sides: value is 5

1: Roll die
2: Get new die
3: Quit
> 2
Sides: 20

1: Roll die
2: Get new die
3: Quit
> 1
20 sides: value is 8

1: Roll die
2: Get new die
3: Quit
> 3

Vedant@LAPTOP-Q7B9365O MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/dice
```

I learned how to create a class with fixed properties (number of sides) and dynamic behavior (rolling). It helped me practice encapsulation and random number generation. I also saw how object-oriented design allows simple real-world concepts to be modeled effectively in code.

19/09/2025 - Coding[Field Guide → Part 2 → 7. Member function → Activities → Test yourself → Dynamic menu]

**Description:**

If we show a menu of options, then we have a switch or set of if statements to select which action to perform. Adding a new action required us to adjust the output as well as the code to select and run the action.

For example, we may want something like this:

```
1: Say Hello
2: Simple Math
-1: Quit
Option:
```

The user then enters the number of the action they want to perform.

With objects, inheritance, and polymorphism we can now replace this with a list of “action” objects. If each action can output itself to the terminal, then we can assign each a number and use that number to select the action object we tell to run when the user chooses it.

**My code:**

```
#include "splashkit.h"

/**
 * @brief Prompt the user and return a string.
 *
 * @param prompt The message displayed to the user
 * @return The string entered by the user
 */
string read_string(string prompt)
{
    write(prompt);
    return read_line();
}

/**
 * @brief Prompt the user and return an integer.
 *
 * Keeps asking until a valid integer is entered.
 *
 * @param prompt The message displayed to the user
 * @return The integer entered by the user
 */
int read_integer(string prompt){
    string line = read_string(prompt);

    while (!is_integer(line))
    {
        write_line("Please enter a integer number.");
        line = read_string(prompt);
    }
    return stoi(line);
}
```

```cpp
/**
 * @brief Abstract base class for all user actions.
 *
 * Defines the interface for showing menu options and executing actions.
 */
class UserAction
{
public:
    virtual ~UserAction() {}
    virtual void show(int optionNumber) const = 0;
    virtual void run() = 0;
    virtual bool selectable() const { return true; }
};


/**
 * @brief A non-selectable divider used for menu formatting.
 */
class DividerAction : public UserAction
{
    string text;

public:
    DividerAction(string t = "----------------") : text(t) {}
    void show(int) const override
    {
        write_line("   " + text);
    }
    void run() override {} // does nothing
    bool selectable() const override { return false; }
};
```

```cpp
/**
 * @brief Action: Prompt the user's name and greet them.
 */
class SayHelloAction : public UserAction
{
public:
    void show(int optionNumber) const override
    {
        write_line(std::to_string(optionNumber) + ": Say Hello");
    }

    void run() override
    {
        string name = read_string("Enter your name: ");
        write_line("Hello, " + name + "! Nice to meet you.");
    }
};
```

```cpp
/**
 * @brief Action: Simple math quiz (addition).
 */
class SimpleMathAction : public UserAction
{
public:
    void show(int optionNumber) const override
    {
        write_line(std::to_string(optionNumber) + ": Simple Math Test");
    }

    void run() override
    {
        int a = rnd(1, 10);
        int b = rnd(1, 10);
        int answer = read_integer("What is " + std::to_string(a) + " + " + std::to_string(b) + "? ");
        if (answer == a + b)
            write_line("Correct!");
        else
            write_line("Oops, the correct answer was " + std::to_string(a + b));
    }
};
```

```cpp
/**
 * @brief Action: Guess the roll of a die.
 */
class GuessDieRollAction : public UserAction
{
public:
    void show(int optionNumber) const override
    {
        write_line(std::to_string(optionNumber) + ": Guess the Die Roll");
    }

    void run() override
    {
        int roll = rnd(1, 6);
        int guess = read_integer("Guess the die roll (1-6): ");
        if (guess == roll)
            write_line("Wow! You guessed it right!");
        else
            write_line("Sorry, the die rolled " + std::to_string(roll) + ".");
    }
};
```

```cpp
/**
 * @brief A dynamic array class for storing UserAction pointers.
 *
 * Automatically resizes when capacity is exceeded.
 */
class ActionArray
{
private:
    UserAction **data;
    int capacity;
    int length;

    void resize()
    {
        int newCap = capacity * 2;
        UserAction **newData = new UserAction *[newCap];
        for (int i = 0; i < length; i++)
            newData[i] = data[i];
        delete[] data;
        data = newData;
        capacity = newCap;
    }

public:
    ActionArray(int initialCap = 4)
    {
        capacity = initialCap;
        length = 0;
        data = new UserAction *[capacity];
    }
```

```cpp
    }

    ~ActionArray()
    {
        for (int i = 0; i < length; i++)
            delete data[i];
        delete[] data;
    }

    void add(UserAction *action)
    {
        if (length == capacity)
            resize();
        data[length++] = action;
    }

    int size() const { return length; }

    UserAction *get(int index) const { return data[index]; }
};
```

```
/**
 * @brief Show a menu of actions and let the user choose and run them.
 *
 * @param actions Reference to an ActionArray containing menu actions
 */
void choose_and_run(ActionArray &actions)
{
    while (true)
    {
        write_line("\n=== Menu ===");
        int optionNumber = 1;

        int mapSize = actions.size();
        int *indexMap = new int[mapSize];
        int mapCount = 0;

        for (int i = 0; i < actions.size(); i++)
        {
            if (actions.get(i)->selectable())
            {
                actions.get(i)->show(optionNumber);
                indexMap[mapCount++] = i;
                optionNumber++;
            }
            else
            {
                actions.get(i)->show(-1);
            }
        }

        write_line("-1: Quit");
        int choice = read_integer("Option: ");

        if (choice == -1)
        {
            delete[] indexMap;
            break;
        }

        if (choice > 0 && choice <= mapCount)
        {
            actions.get(indexMap[choice - 1])->run();
        }
        else
        {
            write_line("Invalid option, try again.");
        }

        delete[] indexMap;
    }
}
```

```
/**
 * @brief Program entry point.
 */
int main()
{
    ActionArray actions;
    actions.add(new SayHelloAction());
    actions.add(new SimpleMathAction());
    actions.add(new DividerAction("=== Games ==="));
    actions.add(new GuessDieRollAction());

    choose_and_run(actions);

    write_line("Goodbye!");
    return 0;
}
```

This task showed me how inheritance and polymorphism can be applied to build flexible systems. By turning actions into objects, I could add new features without rewriting large portions of code. It taught me the power of designing extensible programs using OOP principles.

## 20/09/2025 - Coding[Convert P8 test yourself to dynamic array]

**My code:**

```cpp
#include "splashkit.h"
#include "d_arr_v2.hpp"
#include <string>

const int SCREEN_WIDTH = 800;
const int SCREEN_HEIGHT = 600;

const int PLAYER_RADIUS = 20;
const int PLAYER_SPEED = 3;

/**
 * The rock_data structure stores the information for a single falling rock.
 */
struct rock_data
{
    point_2d pos;
    double size;
    double speed;
    bool active;
};

/**
 * The game_data structure stores all the information needed
 * to manage and update the game state.
 */
struct game_data
{
    dynamic_array<rock_data> rocks;
    point_2d player_pos;
    int score = 0;
    int lives = 3;
    unsigned int next_rock_time = 0;

    game_data() : rocks(10, rock_data()) {} // start with capacity 10
};
```

```
/**
 * @brief Creates and returns a new rock with random size, position, and speed.
 */
rock_data new_rock()
{
    rock_data r;
    r.size = rnd(20, 200);
    r.pos.x = rnd(SCREEN_WIDTH - r.size);
    r.pos.y = -r.size;
    r.speed = rnd(1, 5);
    r.active = true;
    return r;
}

/**
 * @brief Adds a new rock to the game.
 *
 * Increments the score and sets the timer for the next rock spawn.
 *
 * @param game The game state to update
 */
void add_rock(game_data &game)
{
    game.rocks.add(new_rock());
    game.score++;
    game.next_rock_time = current_ticks() + rnd(1000, 6000);
}
```

```
/**
 * @brief Handles keyboard input for moving the player.
 *
 * Left/right arrow keys or A/D keys move the player horizontally.
 *
 * @param game The game state to update
 */
void handle_input(game_data &game)
{
    if (key_down(LEFT_KEY) || key_down(A_KEY))
    {
        game.player_pos.x -= PLAYER_SPEED;
        if (game.player_pos.x < PLAYER_RADIUS)
            game.player_pos.x = PLAYER_RADIUS;
    }
    if (key_down(RIGHT_KEY) || key_down(D_KEY))
    {
        game.player_pos.x += PLAYER_SPEED;
        if (game.player_pos.x > SCREEN_WIDTH - PLAYER_RADIUS)
            game.player_pos.x = SCREEN_WIDTH - PLAYER_RADIUS;
    }
}
```

```
/**
 * @brief Checks collision between a rock and the player.
 *
 * Uses circle-to-circle intersection.
 *
 * @param r The rock to check
 * @param game The current game state (player position)
 * @return true if the rock collides with the player, false otherwise
 */
bool check_collision(const rock_data &r, const game_data &game)
{
    circle rock_circle = circle_at(r.pos.x + r.size / 2, r.pos.y + r.size / 2, r.size / 2);
    circle player_circle = circle_at(game.player_pos.x, game.player_pos.y, PLAYER_RADIUS);
    return circles_intersect(rock_circle, player_circle);
}
```

```
/**
 * @brief Updates the game state.
 *
 * Spawns rocks at random intervals, moves rocks downward,
 * checks collisions, and adjusts score/lives accordingly.
 *
 * @param game The game state to update
 */
void update_game(game_data &game)
{
    if (current_ticks() > game.next_rock_time)
    {
        add_rock(game);
    }

    for (unsigned int i = 0; i < game.rocks.size; i++)
    {
        rock_data &r = game.rocks[i];
        if (!r.active) continue;

        r.pos.y += r.speed;

        if (r.pos.y > SCREEN_HEIGHT)
        {
            game.score += (int)r.size;
            r.active = false;
        }
        else if (check_collision(r, game))
        {
            game.lives--;
            r.active = false;
        }
    }
}

```

```
/**
 * @brief Draws the game state.
 *
 * Clears the screen, draws the score and lives,
 * renders the player, and all active rocks.
 *
 * @param game The game state to render
 */
void draw_game(const game_data &game)
{
    clear_screen(COLOR_WHITE);

    draw_text("Score: " + std::to_string(game.score), COLOR_BLACK, 10, 10);
    draw_text("Lives: " + std::to_string(game.lives), COLOR_BLACK, 10, 40);

    fill_circle(COLOR_BLUE, game.player_pos.x, game.player_pos.y, PLAYER_RADIUS);

    for (unsigned int i = 0; i < game.rocks.size; i++)
    {
        const rock_data &r = game.rocks[i];
        if (r.active)
            fill_circle(COLOR_GRAY, r.pos.x, r.pos.y, r.size / 2);
    }

    refresh_screen(60);
}
```

```
/**
 * @brief Program entry point.
 *
 * Initializes the game, runs the game loop until
 * player loses all lives or quits.
 */
int main()
{
    open_window("Rock Dodge Game", SCREEN_WIDTH, SCREEN_HEIGHT);

    game_data game;
    game.player_pos = point_at(SCREEN_WIDTH / 2, SCREEN_HEIGHT - 50);
    game.next_rock_time = current_ticks() + rnd(1000, 6000);

    while (!quit_requested() && game.lives > 0)
    {
        process_events();
        handle_input(game);
        update_game(game);
        draw_game(game);
    }

    close_window("Rock Dodge Game");
    return 0;
}
```

Here I learned how to refactor existing code to use a dynamic array instead of fixed storage. This reinforced how dynamic memory allows programs to adapt to

user needs, and I also gained experience in debugging issues that arise when migrating from static to dynamic data structures.

## 20/09/2025 - Coding[Small linked list program]

**Description:**

Create your own linked list, and build a small test program using a list of integers. Let the user:

a. See values in the list.
b. Add new values to the end of the list.
c. Delete a value from the list.
d. Insert a value somewhere in the list.

**My code:**

```
#include <stdio.h>
#include <stdlib.h>
#include <new>

/**
 * @brief A node in the linked list.
 */
template <typename T>
struct node
{
    node *next;
    T data;
};

/**
 * @brief A singly linked list structure.
 */
template <typename T>
struct linked_list
{
    node<T> *first;
    node<T> *last;
};

/**
 * @brief Create a new empty linked list.
 */
template <typename T>
linked_list<T> *new_linked_list()
{
    linked_list<T> *list = (linked_list<T> *)malloc(sizeof(linked_list<T>));
    list->first = nullptr;
    list->last = nullptr;
    return list;
}
```

```
/**
 * @brief Add a new node with the given data to the end of the list.
 */
template <typename T>
void add_node(linked_list<T> *list, T data){
    node<T> *new_node = (node<T> *)malloc(sizeof(node<T>));
    new (&new_node->data) T();
    new_node->data = data;
    new_node->next = nullptr;

    if (list->first == nullptr){
        list->first = new_node;
        list->last = new_node;
    }
    else{
        list->last->next = new_node;
        list->last = new_node;
    }
}

/**
 * @brief Print all values in the list.
 */
template <typename T>
void print_list(linked_list<T> *list){
    node<T> *current = list->first;
    while (current != nullptr)
    {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}
```

```
/**
 * @brief Delete the entire linked list and free its memory.
 */
template <typename T>
void delete_linked_list(linked_list<T> *list)
{
    node<T> *current = list->first;
    while (current != nullptr)
    {
        node<T> *next = current->next;
        current->data.~T();
        free(current);
        current = next;
    }

    free(list);
}
```

```cpp
/**
 * @brief Remove a node by value (first match).
 */
template <typename T>
void delete_value(linked_list<T> *list, T value)
{
    node<T> *current = list->first;
    node<T> *previous = nullptr;

    while (current != nullptr && current->data != value)
    {
        previous = current;
        current = current->next;
    }

    if (current == nullptr) return; // not found

    if (previous == nullptr)
    {
        // deleting first node
        list->first = current->next;
    }
    else
    {
        previous->next = current->next;
    }

    if (current == list->last)
    {
        list->last = previous;
    }

    current->data.~T();
    free(current);
}
```

```
/**
 * @brief Insert a value at a given position (0 = beginning).
 */
template <typename T>
void insert_at(linked_list<T> *list, T value, int position)
{
    node<T> *new_node = (node<T> *)malloc(sizeof(node<T>));
    new (&new_node->data) T();
    new_node->data = value;
    new_node->next = nullptr;

    if (position <= 0 || list->first == nullptr)
    {
        // Insert at beginning
        new_node->next = list->first;
        list->first = new_node;
        if (list->last == nullptr) list->last = new_node;
        return;
    }

    node<T> *current = list->first;
    int index = 0;
    while (current->next != nullptr && index < position - 1)
    {
        current = current->next;
        index++;
    }

    new_node->next = current->next;
    current->next = new_node;

    if (new_node->next == nullptr)
    {
        list->last = new_node;
    }
}
```

```c
/**
 * @brief Demonstration of linked list usage with a menu.
 */
int main()
{
    linked_list<int> *list = new_linked_list<int>();
    int choice, value, pos;

    do
    {
        printf("\n=== Linked List Menu ===\n");
        printf("1. View values\n");
        printf("2. Add value to end\n");
        printf("3. Delete value\n");
        printf("4. Insert value at position\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
            print_list(list);
            break;

        case 2:
            printf("Enter value to add: ");
            scanf("%d", &value);
            add_node(list, value);
            break;
```

```c

        case 3:
            printf("Enter value to delete: ");
            scanf("%d", &value);
            delete_value(list, value);
            break;

        case 4:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            printf("Enter position (0 = start): ");
            scanf("%d", &pos);
            insert_at(list, value, pos);
            break;

        case 5:
            delete_linked_list(list);
            printf("Exiting...\n");
            break;

        default:
            printf("Invalid choice!\n");
        }
    } while (choice != 5);

    return 0;
}

```

**Output:**

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/small_linked_list
$ ./sll

=== Linked List Menu ===
1. View values
2. Add value to end
3. Delete value
4. Insert value at position
5. Exit
Enter choice: 1
NULL

=== Linked List Menu ===
1. View values
2. Add value to end
3. Delete value
4. Insert value at position
5. Exit
Enter choice: 2
Enter value to add: 12

=== Linked List Menu ===
1. View values
2. Add value to end
3. Delete value
4. Insert value at position
5. Exit
Enter choice: 2
Enter value to add: 15

=== Linked List Menu ===
1. View values
2. Add value to end
3. Delete value
4. Insert value at position
5. Exit
Enter choice: 2
Enter value to add: 22
```

```
=== Linked List Menu ===
1. View values
2. Add value to end
3. Delete value
4. Insert value at position
5. Exit
Enter choice: 1
12 -> 15 -> 22 -> NULL

=== Linked List Menu ===
1. View values
2. Add value to end
3. Delete value
4. Insert value at position
5. Exit
Enter choice: 4
Enter value to insert: 2
Enter position (0 = start): 1

=== Linked List Menu ===
1. View values
2. Add value to end
3. Delete value
4. Insert value at position
5. Exit
Enter choice: 4
Enter value to insert: 12
Enter position (0 = start): 3

=== Linked List Menu ===
1. View values
2. Add value to end
3. Delete value
4. Insert value at position
5. Exit
Enter choice: 1
12 -> 2 -> 15 -> 12 -> 22 -> NULL
```

From this, I learned how to implement a linked list from scratch and support operations like insertion, deletion, and traversal. This hands-on task solidified my theoretical understanding of pointers and helped me practice manipulating memory manually.

Brief Summary of Concepts

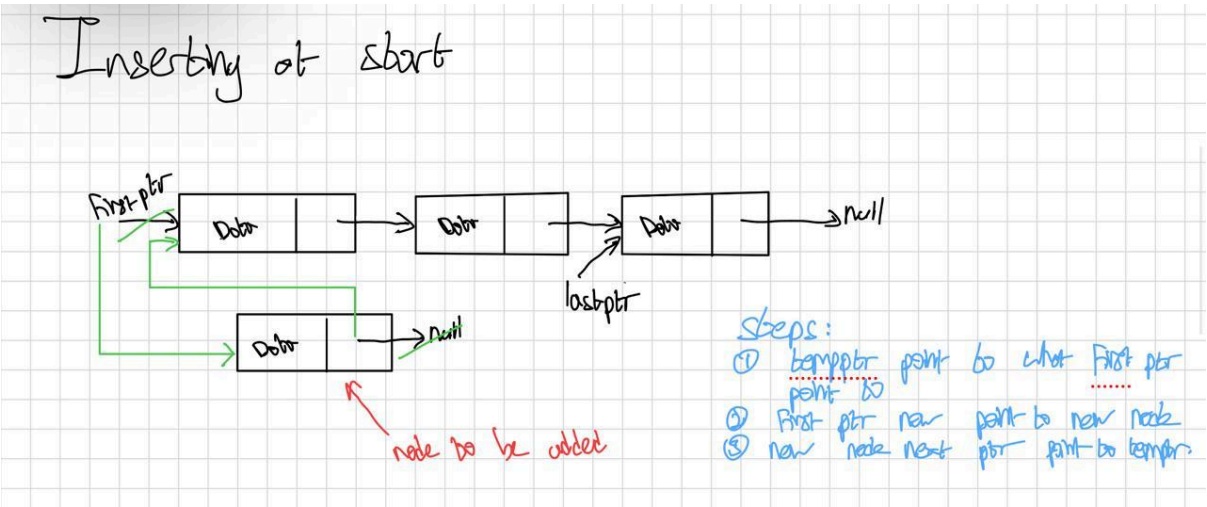
| Concept / Commands | Key Idea / Concept |
|---|---|
| Stack | Memory region for automatic/local variables. Managed by the compiler. Fast allocation/deallocation (LIFO). Automatically cleaned up when functions return. |
| Heap | Memory region for dynamic allocation. Managed manually by the programmer. Slower but flexible. Persists until freed. |
| Malloc / new / new [] | Allocate memory on the heap.<br>- malloc(size) (C) allocates raw memory, returns void*.<br>- new Type (C++) allocates and constructs a single object.<br>- new Type[n] allocates and constructs an array of objects. |
| Realloc | Resizes an already allocated memory block on the heap. Can move data to a new location if needed. |
| Free / delete / delete [] | Deallocate memory from the heap.<br>- free(ptr) releases memory allocated by malloc/realloc/calloc.<br>- delete ptr destroys one object allocated by new.<br>- delete[] ptr destroys an array allocated by new[]. |

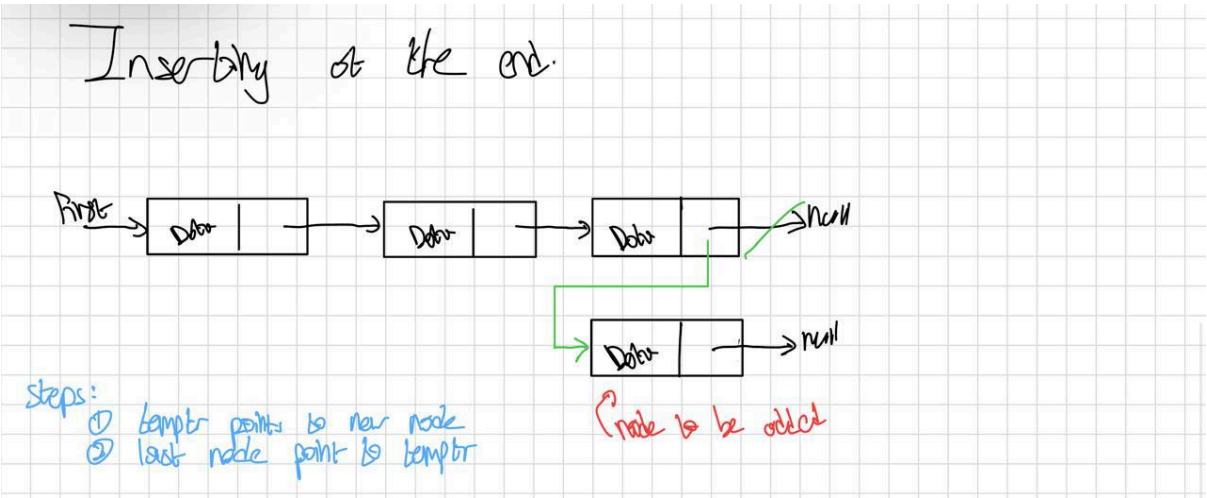
## Memory allocation

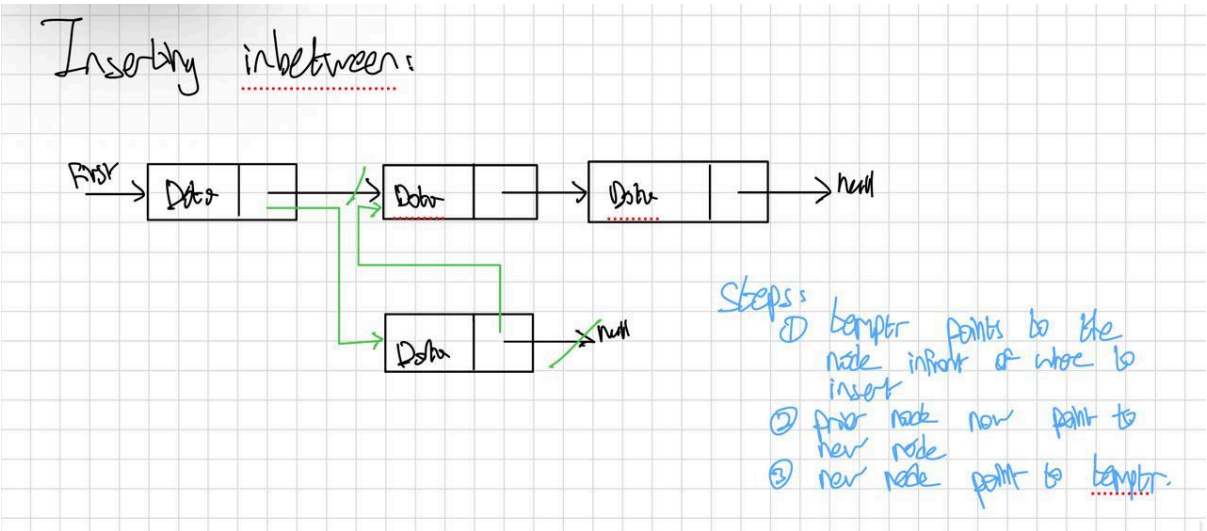
```
#include <cstdlib>
#include <string>

using std::string;

struct person_data
{
    string name;
    int age;
};

int main()
{
    int *arr;
    arr = (int *) malloc(10 * sizeof(int));

    if (arr == NULL)
    {
        return 1;
    }

    int *new_arr;
    new_arr = (int *) realloc(arr, 20 * sizeof(int));

    arr = new_arr;

    free(arr);

    person_data *ptr;

    ptr = new person_data();
    ptr->name = "Jane Doe";
    ptr->age = 30;

    return 0;
}
```

| | Response |
|---|---|
| 1: Explain how arr and malloc are working here, and how this relates to the heap? | arr is a pointer variable stored on the stack. malloc requests a block of memory from the heap, big enough to hold 10 integers. The memory address returned by malloc (on the heap) is stored in arr. So, the pointer itself is on the stack, but the actual integers live on the heap. |
| 2: Why is the cast needed here? | In C, malloc returns void*, which can be assigned to any pointer type without casting, but in C++, the cast is required because implicit conversion from void* to another pointer type is not allowed. That's why (int*) is used here. |
| 3: What is sizeof and why are we multiplying it here? | Sizeof gives the number of bytes required for a type.<br><br>10 * sizeof(int) ensures we allocate exactly enough memory for 10 integers, even if int size changes on another platform.This makes the code portable and correct. |
| 4: Why test arr here? | If the system cannot allocate memory (out of heap space), malloc returns NULL. Checking prevents using an invalid pointer, which would cause crashes. |
| 5: Explain what occurs with realloc here and why new_arr is needed. | realloc(arr, 20 * sizeof(int)) resizes the block to fit 20 integers. If possible, it expands the existing block; otherwise, it allocates a new block and copies the old contents over.<br><br>We use new_arr as a temporary pointer in case realloc fails (returns NULL). This avoids losing the original arr. |
| 6: Explain what occurs here when free is called. | Frees the memory previously allocated by malloc (or realloc). After this, that memory is returned to the system and is no longer valid to access. |
| 7: Why should we use new rather than malloc here? What does it do? | new<br><br>It allocates memory on the heap. It also calls the constructor of the object (person_data() here).<br><br>malloc would only reserve raw memory and not initialize the object (e.g., std::string name would break). |
| 8: What do the -> symbols mean here? How would you use this? | The → operator is shorthand for dereferencing a pointer to an object/struct and accessing its members. |

## Illustrate insert in your linked list



# Reflection

## What is the most important thing you learned from this and why?

The most important thing I learned was how memory management works at a low level in C++. Understanding the stack vs. heap, and practicing with malloc, free, new, delete, and pointers gave me a much deeper appreciation of how programs actually run. This is crucial knowledge for writing efficient, safe, and scalable software.

## What challenges and opportunities do you see with dynamic memory management?

The main challenge is avoiding bugs such as memory leaks, dangling pointers, or segmentation faults, which can be very difficult to debug. However, the opportunity is that with proper understanding and practice, dynamic memory management allows programs to be highly flexible and efficient, handling large datasets and complex structures in ways that static memory cannot.

# References

## Generative AI acknowledgement

No AI was used for either the code or this document.